

Exercise 1

Consider three periodic tasks τ_1 , τ_2 , and τ_3 (having decreasing priority), which share three resources, A , B , and C . The longest duration $d_i(R)$ for a task τ_i on resource R is given in the following table (there are no nested critical sections):

process i	A	B	C
τ_1	2	0	2
τ_2	2	3	0
τ_3	3	2	5

Assume that the resources are accessed with

- the Priority Inheritance Protocol, or
- the Priority Ceiling Protocol,

and compute the maximum blocking time B_i for each task.

a) $\text{prio}(\tau_1) > \text{prio}(\tau_2) > \text{prio}(\tau_3)$ ☐ direct blocking ☐ push-through blocking

process i	A	B	C
τ_1	2	0	2
τ_2	2	3	0
τ_3	3	2	5

CS's that

could block: $\{A_2, A_3, C_3\}$
 $B_1 = 2 + 5 = 7$

process i	A	B	C
τ_1	2	0	2
τ_2	2	3	0
τ_3	3	2	5

CS's that

could block: $\{A_3, B_3, C_3, \cancel{A_2}\}$
 $B_2 = 5$

process i	A	B	C
τ_1	2	0	2
τ_2	2	3	0
τ_3	3	2	5

CS's that

could block: $\{\}$
 $B_3 = 0$

Priority Inheritance: A task can be blocked for one critical section by each lower priority task

Priority Ceiling Protocol: A task can be blocked at most for one critical section by one lower priority task

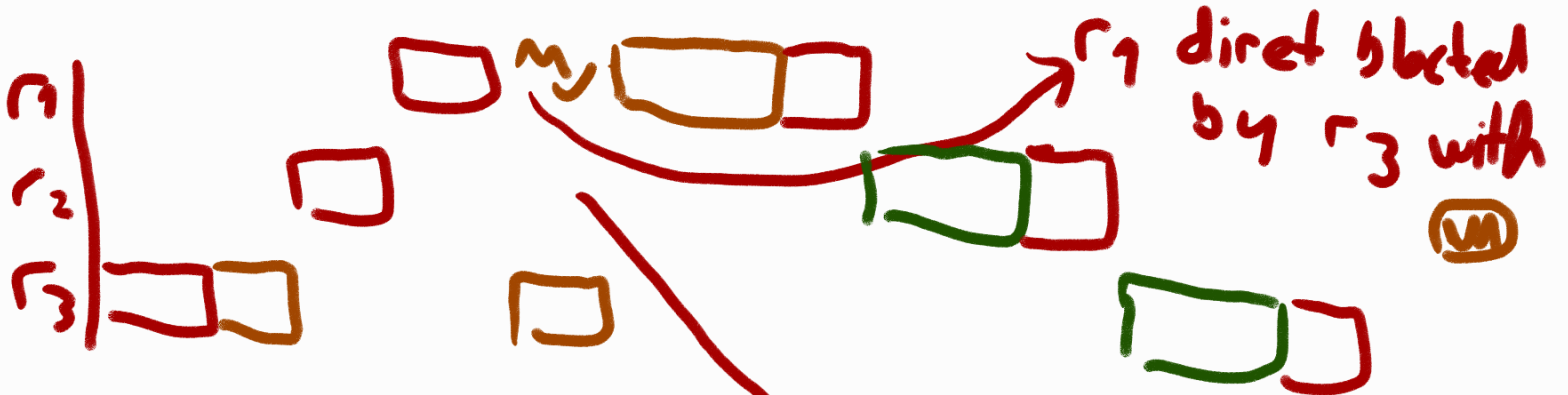
b) $B_1 = 5$ $B_2 = 5$ $B_3 = 0$

Two kinds of blocking of a higher-priority job:

- **Direct blocking:** Higher-priority job tries to enter critical section held by lower-priority job.
 - Needed to ensure mutual exclusion.
- **Push-through blocking:** Medium-priority job blocked by lower-priority job that has inherited a higher priority from a job it blocks directly.
 - Needed to avoid unbounded priority inversion!

For both PI and PC

by PI and PC $\text{prio}(r_1) > \text{prio}(r_2) > \text{prio}(r_3)$



r_1	(2)	0	$\{ \text{[resource icon]} \}$
r_2	0	(2)	$\{ \text{[resource icon]} \}$
r_3	(2)	(2)	$\{ \text{[resource icon]} \}$

r_2 push through blocked by r_3 with [resource icon]

over r_1 which is directly blocked by r_3

- bottom no direct blocking poss.
- top no push through blocking poss.

Priority Inheritance

- Prevents Priority Inversion
- But

Priority Ceiling

- Prevents Chain Blocking

(only works if r_1 and r_3 share a resource)



Exercise 2

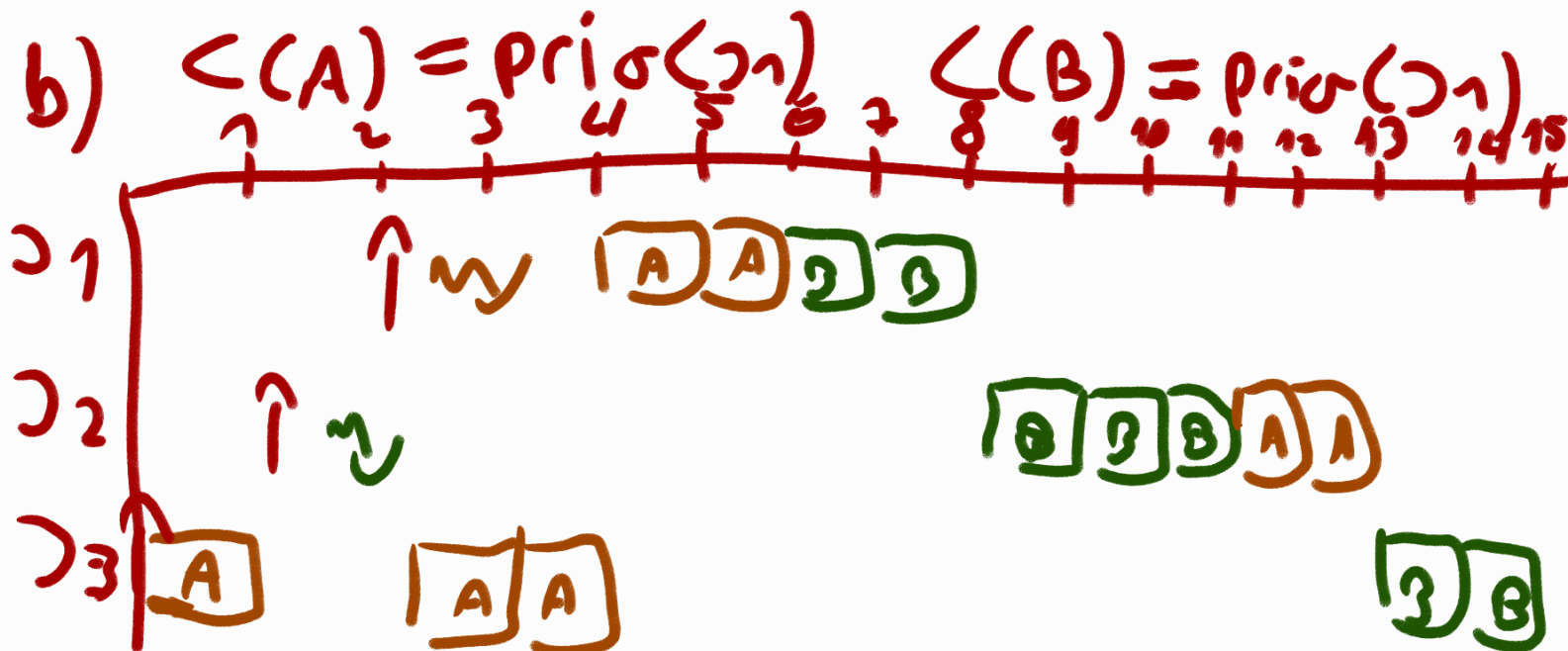
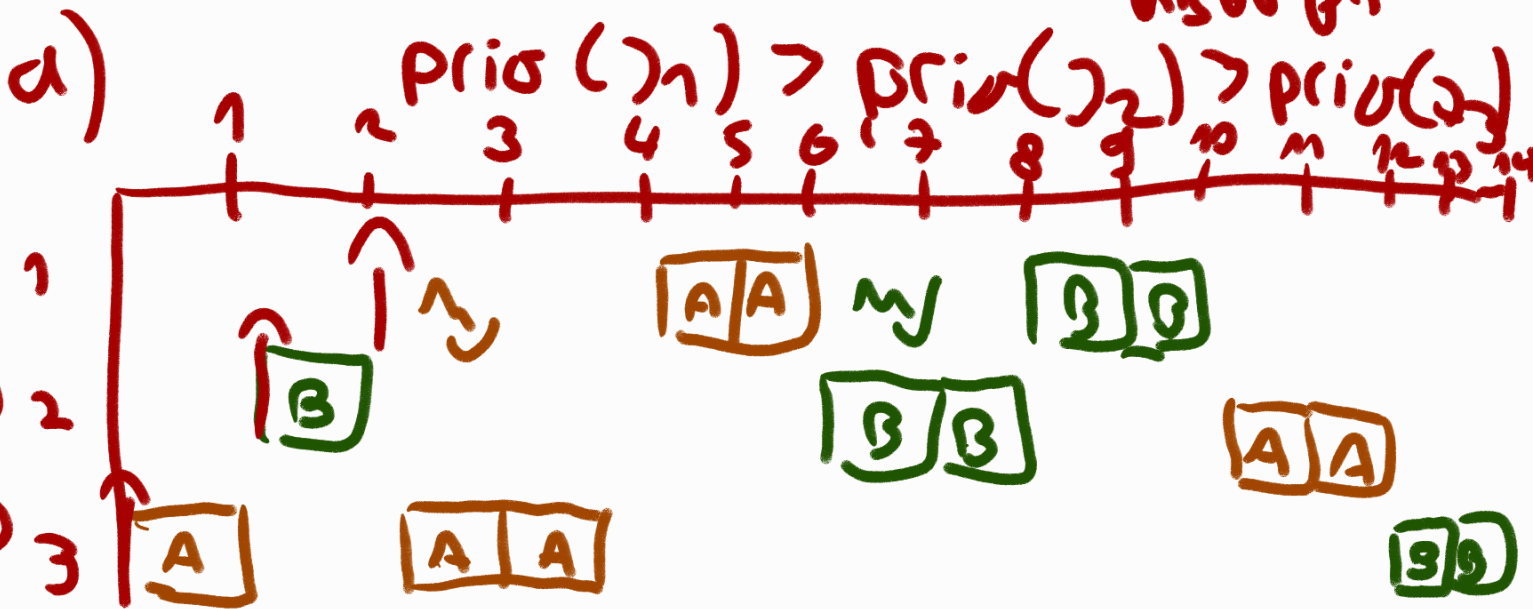
Assume three aperiodic tasks J_1 , J_2 , and J_3 (having decreasing priority), which share two resources, A and B . The demanded duration $d_i(R)$ for a task J_i on resource R is given in the following table:

process i	A	B
J_1	2	2
J_2	2	3
J_3	3	2

a) Assume there are no overlapping (and no nested) critical sections. Find arrival times (and blocks of normal executions if necessary) such that J_1 suffers chained blocking by J_2 and J_3 when scheduled by the priority inheritance protocol.

b) Now, schedule your tasks from a) with the priority ceiling protocol.

low prio
task while
ex. critical
section
preempted by
higher prio



- Priority Ceiling? ✓
- way to no push through ✓
- blocking? coz no higher prio task
- how push through by ^{prior} task
- PI? (2)

drain block-in style task

- scheduler active as soon as lower prio task finishes it's critical

For CCSA) just section

look at line in table what is highest prio task that will request this resource

when entering critical section:

- online consider all semaphores currently locked by jobs other than current. From all ceilings of ^{these} semaphores take maximal one

→ This way never more than 1 task holding semaphores that are able to block same higher prio task

why (2)